

DB 쿼리 감사 로그 시스템 — Superset + PostgreSQL 전용 사용법

****대상 환경****: Apache Superset → PostgreSQL (Azure Cloud / RedHat 9.7, 비암호화 연결)

1. 시스템 개요

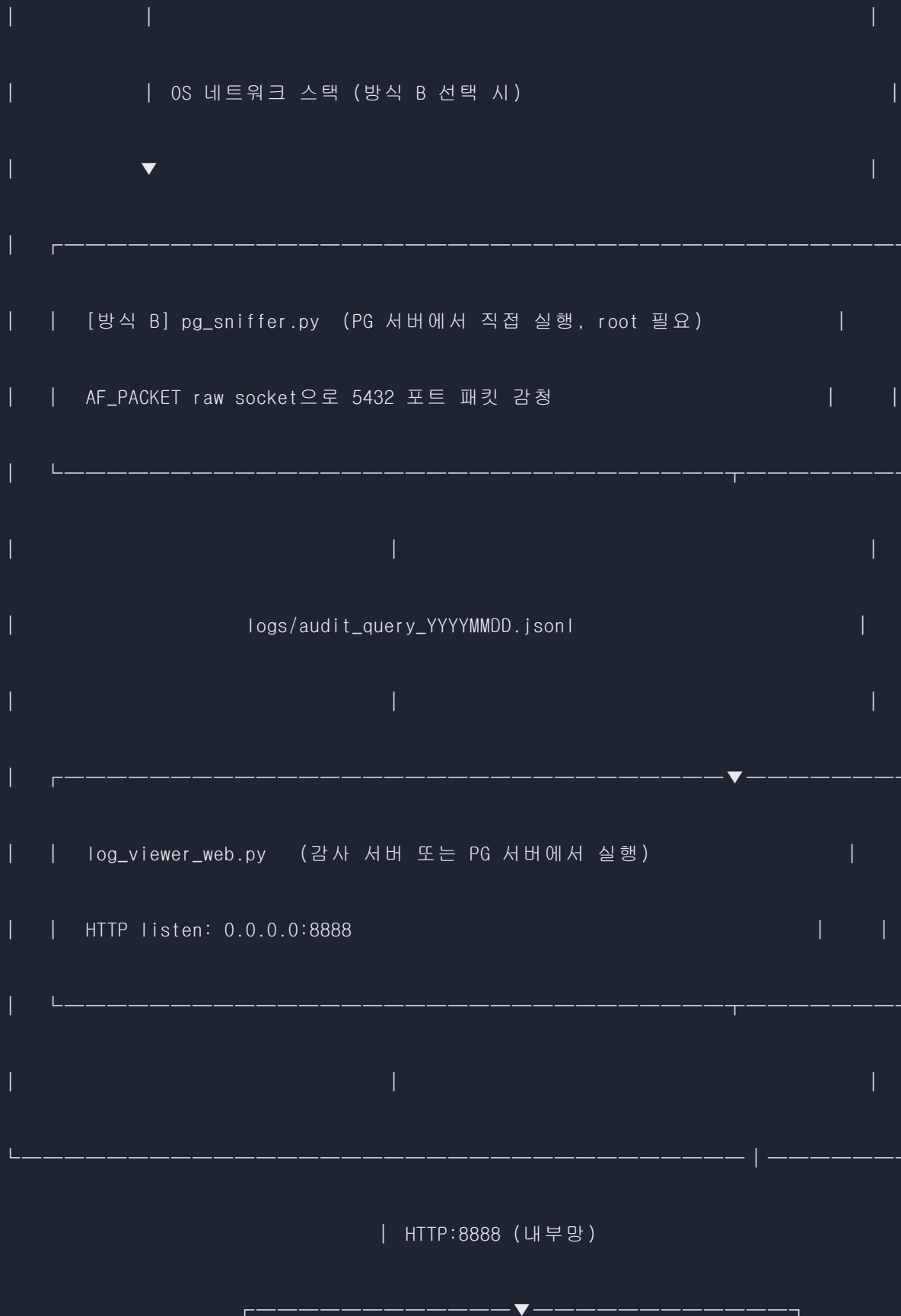
Apache Superset이 PostgreSQL에 실행하는 모든 쿼리를 **6하원칙(누가/언제/어디서/무엇을/어떻게/왜)**에 따라 자동으로 로그 파일에 기록한다.

항목	내용
감사 대상	Superset → PostgreSQL 모든 SQL (SELECT / DML / DDL)
암호화 전제	Superset ↔ PostgreSQL 간 **비암호화(sslmode=disable)**
DB 서버 OS	Azure Cloud / RedHat 9.7
외부 패키지	**없음** — Python 표준 라이브러리만 사용
로그 형식	JSON Lines (.jsonl), 일별 파일, SHA-256 무결성 해시
조회 방법	웹 브라우저(포트 8888) 또는 CLI

2. 아키텍처

2.1 전체 구성도

[패킷 캡처 결과]			
[패킷 1: TCP Reset (RST)]			
[패킷 2: TCP Reset (RST)]			
[패킷 3: TCP Reset (RST)]			
[패킷 4: TCP Reset (RST)]			
[패킷 5: TCP Reset (RST)]			
[패킷 6: TCP Reset (RST)]			
[패킷 7: TCP Reset (RST)]			
[패킷 8: TCP Reset (RST)]			
[패킷 9: TCP Reset (RST)]			
[패킷 10: TCP Reset (RST)]			
[패킷 11: TCP Reset (RST)]			
[패킷 12: TCP Reset (RST)]			
[패킷 13: TCP Reset (RST)]			
[패킷 14: TCP Reset (RST)]			
[패킷 15: TCP Reset (RST)]			
[패킷 16: TCP Reset (RST)]			
[패킷 17: TCP Reset (RST)]			
[패킷 18: TCP Reset (RST)]			
[패킷 19: TCP Reset (RST)]			
[패킷 20: TCP Reset (RST)]			
[패킷 21: TCP Reset (RST)]			
[패킷 22: TCP Reset (RST)]			
[패킷 23: TCP Reset (RST)]			
[패킷 24: TCP Reset (RST)]			
[패킷 25: TCP Reset (RST)]			
[패킷 26: TCP Reset (RST)]			
[패킷 27: TCP Reset (RST)]			
[패킷 28: TCP Reset (RST)]			
[패킷 29: TCP Reset (RST)]			
[패킷 30: TCP Reset (RST)]			
[패킷 31: TCP Reset (RST)]			
[패킷 32: TCP Reset (RST)]			
[패킷 33: TCP Reset (RST)]			
[패킷 34: TCP Reset (RST)]			
[패킷 35: TCP Reset (RST)]			
[패킷 36: TCP Reset (RST)]			
[패킷 37: TCP Reset (RST)]			
[패킷 38: TCP Reset (RST)]			
[패킷 39: TCP Reset (RST)]			
[패킷 40: TCP Reset (RST)]			
[패킷 41: TCP Reset (RST)]			
[패킷 42: TCP Reset (RST)]			
[패킷 43: TCP Reset (RST)]			
[패킷 44: TCP Reset (RST)]			
[패킷 45: TCP Reset (RST)]			
[패킷 46: TCP Reset (RST)]			
[패킷 47: TCP Reset (RST)]			
[패킷 48: TCP Reset (RST)]			
[패킷 49: TCP Reset (RST)]			
[패킷 50: TCP Reset (RST)]			
[패킷 51: TCP Reset (RST)]			
[패킷 52: TCP Reset (RST)]			
[패킷 53: TCP Reset (RST)]			
[패킷 54: TCP Reset (RST)]			
[패킷 55: TCP Reset (RST)]			
[패킷 56: TCP Reset (RST)]			
[패킷 57: TCP Reset (RST)]			
[패킷 58: TCP Reset (RST)]			
[패킷 59: TCP Reset (RST)]			
[패킷 60: TCP Reset (RST)]			
[패킷 61: TCP Reset (RST)]			
[패킷 62: TCP Reset (RST)]			
[패킷 63: TCP Reset (RST)]			
[패킷 64: TCP Reset (RST)]			
[패킷 65: TCP Reset (RST)]			
[패킷 66: TCP Reset (RST)]			
[패킷 67: TCP Reset (RST)]			
[패킷 68: TCP Reset (RST)]			
[패킷 69: TCP Reset (RST)]			
[패킷 70: TCP Reset (RST)]			
[패킷 71: TCP Reset (RST)]			
[패킷 72: TCP Reset (RST)]			
[패킷 73: TCP Reset (RST)]			
[패킷 74: TCP Reset (RST)]			
[패킷 75: TCP Reset (RST)]			
[패킷 76: TCP Reset (RST)]			
[패킷 77: TCP Reset (RST)]			
[패킷 78: TCP Reset (RST)]			
[패킷 79: TCP Reset (RST)]			
[패킷 80: TCP Reset (RST)]			
[패킷 81: TCP Reset (RST)]			
[패킷 82: TCP Reset (RST)]			
[패킷 83: TCP Reset (RST)]			
[패킷 84: TCP Reset (RST)]			
[패킷 85: TCP Reset (RST)]			
[패킷 86: TCP Reset (RST)]			
[패킷 87: TCP Reset (RST)]			
[패킷 88: TCP Reset (RST)]			
[패킷 89: TCP Reset (RST)]			
[패킷 90: TCP Reset (RST)]			
[패킷 91: TCP Reset (RST)]			
[패킷 92: TCP Reset (RST)]			
[패킷 93: TCP Reset (RST)]			
[패킷 94: TCP Reset (RST)]			
[패킷 95: TCP Reset (RST)]			
[패킷 96: TCP Reset (RST)]			
[패킷 97: TCP Reset (RST)]			
[패킷 98: TCP Reset (RST)]			
[패킷 99: TCP Reset (RST)]			
[패킷 100: TCP Reset (RST)]			



```
| 사용자 PC 웹 브라우저 |  
  
| http://<서버 IP>:8888 |  
  
└────────────────────────┘
```

2.2 감사 방식 선택

비교 항목	방식 A: pg_proxy.py (권장)	방식 B: pg_sniffer.py
작동 원리	TCP 미들맨 프록시	OS 레벨 패킷 스니퍼
설치 위치	PG 서버 또는 별도 서버	PG 서버 직접 설치 필수
root 권한	**불필요**	root 또는 CAP_NET_RAW 필요
Superset 변경	DB URI 변경 필요	**변경 없음**
PG 서버 변경	**없음**	**없음**
SQL 추출 신뢰도	높음 (TCP 스트림 직접 파싱)	높음 (동일 파서)
SSL 암호화	비암호화 전용 (현재 환경 적합)	비암호화 전용 (현재 환경 적합)

현재 환경(비암호화) 기준: 방식 A(pg_proxy.py) 권장 — root 없이 일반 계정으로 실행 가능.

3. 배포 패키지 — 폐쇄망 반입

3.1 반입 대상 폴더

```

deploy/                                ← 이 폴더 전체를 내부망 서버로 복사

├── config.py                          # ★ 가장 먼저 수정

├── query_logger.py                    # 로그 기록 · 읽기 핵심 엔진

├── pg_proxy.py                        # [방식 A] Superset 감사 프록시

├── pg_sniffer.py                      # [방식 B] 패킷 스니퍼

├── log_viewer_web.py                  # 웹 조회 서버 (포트 8888)

├── log_viewer.py                      # CLI 조회

├── oracle_proxy.py                   # Oracle TNS 프록시 (Superset 환경에서는 미사용)

├── packet_sniffer.py                 # Oracle 패킷 스니퍼 (Superset 환경에서는 미사용)

|

├── run_pg_proxy.bat / .sh            # [방식 A] 런처

├── run_pg_sniffer.bat / .sh          # [방식 B] 런처

├── run_viewer_web.bat / .sh          # 웹 뷰어 런처

├── run_viewer_cli.bat / .sh          # CLI 뷰어 런처

|

├── prepare_offline.ps1 / .sh         # 오프라인 Python 준비 (외부망에서 사전 실행)

├── python/                           # Python embeddable (Windows 전용, 내부 포함)

└── logs/                             # 감사 로그 저장 디렉토리 (자동 생성)

```

Oracle 관련 파일(`oracle_proxy.py`, `packet_sniffer.py`, `run_oracle_sniffer.*`)은 Superset + PostgreSQL 환경에서 사용하지 않는다. 폴더에 함께 들어 있어도 무방하다.

3.2 대상 서버 환경 확인 (반입 전)

RedHat 9.7 서버에서 아래 항목을 미리 확인한다.

1. OS 버전

```
cat /etc/os-release
```

2. Python 설치 여부 (3.8 이상이면 별도 준비 불필요)

```
python3 --version
```

3. root / sudo 권한 확인

```
sudo -l
```

4. SELinux 상태 확인 (Enforcing이면 raw socket)

```
getenforce
```

5. 방화벽 - 8888 포트 오픈 여부

```
sudo firewall-cmd --list-all
```

6. PostgreSQL 포트 확인

```
sudo ss -tlnp | grep 5432
```

7. PostgreSQL SSL 설정 확인 (방식 A · B 모두 ssl)

```
sudo -u postgres psql -c "SHOW ssl;"
```

RedHat 9.7은 기본적으로 Python 3.9 이상이 설치되어 있으므로 `python/` 폴더 없이도 동작한다.

3.3 반입 절차

1단계 — (필요 시만) 외부망에서 Python RPM 준비

RedHat 9.7에 Python이 없는 경우 외부망에서:

```
# RedHat 9 계 열
```

```
dnf download --resolve python3
```

→ *.rpm 파일을 USB에 담아 반입 후 내부망 서버에

```
sudo rpm -ivh *.rpm
```

2단계 — deploy 폴더를 내부망 PG 서버로 복사


```
# USB 또는 내부 파일 전송으로 복사
```

```
scp -r deploy/ audit@<PG서버 IP>:/opt/audit/
```

또는 USB 마운트 후

```
cp -r /media/usb/deploy/ /opt/audit/
```

3단계 — config.py 수정

```
cd /opt/audit
```

```
vi config.py          # PG_HOST, PG_PORT, PG_DB, PG_SCHEMA 수정
```

4단계 — 실행 (방식 선택)

```
# 방식 A: pg_proxy.py (권장)
```

```
bash run_pg_proxy.sh --pg-host 127.0.0.1 --pg-port 5432 --listen-port 5433
```

방식 B: pg_sniffer.py

```
bash run_pg_sniffer.sh --pg-port 5432 --db-name mydb
```

웹 뷰어

```
bash run_viewer_web.sh --host 0.0.0.0 --port 8888
```

4. 설정 파일 (config.py) — 가장 먼저 수정

```
# — PostgreSQL 연결 정보 (pg_proxy.py / pg_sniffer.py 기본값) —————

PG_HOST    = "127.0.0.1"      # ★ 실제 PostgreSQL 서버 IP (PG 서버 직접이면 127.0.0.1)

PG_PORT    = 5432             # PostgreSQL 리스너 포트

PG_DB      = "superset_db"    # ★ 실제 데이터베이스 이름

PG_SCHEMA  = "public"        # ★ 감사할 스키마 (보통 public)
```

로그 저장 경로 (절대경로 권장)

```
LOG_DIR    = Path("/var/log/audit") # ★ 실제 경로로 변경
```

현재 스키마 확인:

```
-- PostgreSQL에서 실행

SELECT schema_name FROM information_schema.schemata ORDER BY 1;

SELECT DISTINCT table_schema FROM information_schema.tables

WHERE table_type = 'BASE TABLE' ORDER BY 1;
```

5. 방식 A: pg_proxy.py (권장)

5.1 개요

Superset과 PostgreSQL 사이에 TCP 프록시를 끼워 넣어 모든 SQL을 투명하게 감청한다. root 권한이 불필요하며, PostgreSQL 서버 설정 변경도 없다.

Superset —TCP:5433—▶ pg_proxy.py —TCP:5432—▶ PostgreSQL

(감사 기록)

5.2 실행

기본 실행 (PG가 같은 서버에 있는 경우)

```
python3 pg_proxy.py W
```

```
--pg-host 127.0.0.1 W      # PostgreSQL 실제 서버 주소
```

```
--pg-port 5432 W          # PostgreSQL 실제 포트
```

```
--listen-port 5433 W      # Superset이 연결할 프록시 포트
```

```
--db-name superset_db W   # 로그에 기록할 DB명
```

```
--db-schema public       # 로그에 기록할 스키마명
```

런처 스크립트 사용

```
bash run_pg_proxy.sh --pg-host 127.0.0.1 --pg-port 5432 --listen-port 5433
```

파라미터	설명	기본값
<code>--listen-host</code>	프록시 바인딩 주소	0.0.0.0
<code>--listen-port</code>	Superset이 접속할 포트	5433
<code>--pg-host</code>	실제 PostgreSQL 서버 IP	config.py PG_HOST
<code>--pg-port</code>	실제 PostgreSQL 포트	5432
<code>--db-name</code>	로그용 DB명	config.py PG_DB
<code>--db-schema</code>	로그용 스키마명	config.py PG_SCHEMA

5.3 Superset DB 연결 URI 변경

Superset 관리 화면에서 DB 연결 URI를 프록시 주소로 변경한다.

변경 전:

```
postgresql+psycpg2://user:password@PG서버IP:5432/superset_db
```

변경 후:

```
postgresql+psycpg2://user:password@프록시서버IP:5433/superset_db?sslmode=disable
```

`sslmode=disable` 을 반드시 명시한다. Superset/SQLAlchemy가 SSL을 시도하면 프록시가 SSL 활성화 감지 후 경고를 남기고 해당 세션의 SQL 추출을 중단한다.

Superset에서 변경하는 방법:

1. Superset 상단 메뉴 → **Settings** → **Database Connections**
2. 대상 PostgreSQL DB 클릭 → **Edit**
3. **SQLAlchemy URI** 필드를 위 "변경 후" 주소로 교체
4. **Test Connection** 클릭으로 연결 확인
5. **Save** 저장

5.4 systemd 서비스 등록 (자동 시작)

```
sudo tee /etc/systemd/system/pg-audit-proxy.service << 'EOF'
```

```
[Unit]
```

```
Description=PostgreSQL Audit Proxy (Superset)
```

```
After=network.target postgresql.service
```

```
[Service]
```

```
ExecStart=/usr/bin/python3 /opt/audit/pg_proxy.py W
```

```
    --pg-host 127.0.0.1 W
```

```
    --pg-port 5432 W
```

```
    --listen-port 5433 W
```

```
    --db-name superset_db W
```

```
    --db-schema public
```

```
WorkingDirectory=/opt/audit
```

```
Restart=always
```

```
RestartSec=5
```

```
User=audituser
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable --now pg-audit-proxy
```

```
sudo systemctl status pg-audit-proxy
```

`User=audituser` 부분을 실제 실행 계정으로 변경한다. root가 아닌 일반 계정으로 실행 가능하다.

6. 방식 B: pg_sniffer.py

6.1 개요

PostgreSQL 서버의 OS 네트워크 스택에서 원시 소켓(raw socket)으로 5432 포트 트래픽을 복사해 감청한다. Superset 설정 변경이 전혀 없지만 **반드시 PG 서버에 직접 설치**해야 하고 **root 권한**이 필요하다.

6.2 실행

```
# root로 직접 실행
```

```
sudo python3 pg_sniffer.py W
```

```
--pg-port 5432 W
```

```
--db-server 10.0.0.5 W      # 로그용 PG 서버 표시 IP
```

```
--db-name superset_db W
```

```
--db-schema public
```

런처 스크립트 사용

```
bash run_pg_sniffer.sh --pg-port 5432 --db-name superset_db
```

파라미터	설명	기본값
<code>--pg-port</code>	감청할 PostgreSQL 포트	5432
<code>--db-server</code>	로그용 서버 명칭	config.py PG_HOST
<code>--db-name</code>	로그용 DB명	config.py PG_DB
<code>--db-schema</code>	로그용 스키마명	config.py PG_SCHEMA

6.3 root 없이 실행 (CAP_NET_RAW)


```
# python3 바이너리에 CAP_NET_RAW capability 부여 (1회만 실행)
```

```
sudo setcap cap_net_raw+ep /usr/bin/python3
```

이후 일반 계정으로 실행 가능

```
python3 pg_sniffer.py --pg-port 5432
```

6.4 systemd 서비스 등록

```
sudo tee /etc/systemd/system/pg-audit-sniffer.service << 'EOF'

[Unit]

Description=PostgreSQL Audit Packet Sniffer (Superset)

After=network.target

[Service]

ExecStart=/usr/bin/python3 /opt/audit/pg_sniffer.py W

    --pg-port 5432 W

    --db-name superset_db W

    --db-schema public

WorkingDirectory=/opt/audit

Restart=always

RestartSec=5

User=root

[Install]

WantedBy=multi-user.target

EOF

sudo systemctl daemon-reload
```

```
sudo systemctl enable --now pg-audit-sniffer
```

```
sudo systemctl status pg-audit-sniffer
```

7. 웹 로그 조회 — 사용자 PC 브라우저에서 접속

7.1 웹 뷰어 서버 기동 (PG 서버 또는 감사 서버에서)

```
# 기본 실행 (모든 IP에서 접속 허용)
```

```
python3 log_viewer_web.py --host 0.0.0.0 --port 8888
```

런처 스크립트 사용

```
bash run_viewer_web.sh --host 0.0.0.0 --port 8888
```

systemd 서비스로 등록 (자동 시작)

```
sudo tee /etc/systemd/system/audit-log-viewer.service << 'EOF'
```

```
[Unit]
```

```
Description=Audit Log Web Viewer
```

```
After=network.target
```

```
[Service]
```

```
ExecStart=/usr/bin/python3 /opt/audit/log_viewer_web.py W
```

```
    --host 0.0.0.0 --port 8888
```

```
WorkingDirectory=/opt/audit
```

```
Restart=always
```

```
RestartSec=5
```

```
User=audituser
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable --now audit-log-viewer
```

7.2 방화벽 설정 (RedHat 9.7 / firewalld)

```
# 사용자 PC IP 대역만 허용 (예: 192.168.10.0/24)
```

```
sudo firewall-cmd --permanent --add-rich-rule='
```

```
rule family="ipv4"
```

```
source address="192.168.10.0/24"
```

```
port protocol="tcp" port="8888"
```

```
accept'
```

```
sudo firewall-cmd --reload
```

설정 확인

```
sudo firewall-cmd --list-rich-rules
```

필요 시 특정 IP 1개만 허용

```
sudo firewall-cmd --permanent --add-rich-rule='
```

```
rule family="ipv4"
```

```
source address="192.168.10.50/32"
```

```
port protocol="tcp" port="8888"
```

```
accept'
```

```
sudo firewall-cmd --reload
```

****주의**:** 인터넷에 노출된 서버에서는 절대로 0.0.0.0:8888을 방화벽에서 전체 허용하지 않는다. 조회 담당자 IP만 허용한다.

7.3 사용자 PC 브라우저 접속

```
http://<PG서버 IP>:8888
```

예시:

```
http://10.0.0.5:8888
```

접속이 안 될 경우 확인:

```
# 서버에서 리스닝 확인
```

```
ss -tlnp | grep 8888
```

사용자 PC에서 연결 테스트

```
telnet 10.0.0.5 8888      # Windows
```

```
nc -zv 10.0.0.5 8888      # Linux/Mac
```

7.4 웹 뷰어 주요 화면

화면	URL	주요 기능
로그 검색	<code>http://서버 IP:8888/</code>	기간·사용자·업무유형·개인정보 포함 여부 필터
상세 조회	<code>http://서버 IP:8888/detail?id=...</code>	6하원칙 전체 + SQL + 결과 데이터
무결성 검증	<code>http://서버 IP:8888/verify</code>	SHA-256 해시 일괄 검증
CSV 내보내기	<code>http://서버 IP:8888/export</code>	BOM UTF-8 CSV 다운로드

개인정보 마스킹 토글

로그 목록/상세 화면 상단의 **개인정보 마스킹** 체크박스로 마스킹/원문을 즉시 전환할 수 있다. 서버 재요청 없이 브라우저에서 JS로 처리된다.

DB 종류 필터

웹 검색 화면에서 `execution_method` 기준으로 감사 방식을 필터할 수 있다:

- `PG Proxy` — `pg_proxy.py` 에서 수집된 로그
- `PG Sniffer` — `pg_sniffer.py` 에서 수집된 로그

8. 로그 파일 스키마

파일 위치: `logs/audit_query_YYYYMMDD.jsonl` (JSON Lines, UTF-8, 200 MB 초과 시 `_001` 분할)


```

{

  "log_id": "uuid4",

  "when": { "start_time": "2026-04-28T09:15:30.123456", "end_time": "...", "duration_ms": 42 },

  "who": { "user_id": "superset_user", "user_name": "superset_user", "user_role": "Superset", "ip": "10.0.0.10" },

  "where": { "client_ip": "10.0.0.10", "hostname": "superset-host",

    "db_server": "10.0.0.5", "db_name": "superset_db", "db_schema": "public" },

  "what": { "query_type": "SELECT", "operation_type": "조회",

    "sql": "SELECT id, name FROM customers WHERE ...",

    "parameters": null, "row_count": 25, "affected_rows": 0 },

  "how": { "execution_method": "PG Proxy", // "PG Proxy" | "PG Sniffer"

    "application": "FinanceApp v1.0.0", "session_id": "a1b2c3d4" },

  "why": { "purpose": "조회", "reference_no": "" },

  "result": {

    "status": "SUCCESS",

    "error_message": null,

    "has_personal_info": false,

    "personal_info_counts": { "주민등록번호": 0, "카드번호": 0, "계좌번호": 0, "전화번호": 0 },

    "result_count": 25,

```

```

    "result_data": ["값1", "값2", "..."]

},

    "integrity_hash": "sha256(log_id+start_time+sql+status)"

}

```

9. 운영 체크리스트

초기 설치

- ☐ `config.py` 수정 완료 (PG_HOST, PG_PORT, PG_DB, PG_SCHEMA, LOG_DIR)
- ☐ `logs/` 디렉토리 쓰기 권한 설정 (`chmod 700 /opt/audit/logs/`)
- ☐ PostgreSQL SSL 확인 (`SHOW ssl;` → off 확인)
- ☐ **방식 A**: `pg_proxy.py` 실행, Superset DB URI 변경, 연결 테스트 성공
- ☐ **방식 B**: `pg_sniffer.py` 실행 (root 또는 CAP_NET_RAW 확인)
- ☐ Superset에서 쿼리 실행 → `logs/*.jsonl` 파일 생성 확인
- ☐ 웹 뷰어 실행 (`run_viewer_web.sh`) → 사용자 PC 브라우저 접속 성공
- ☐ 방화벽 8888 포트 → 조회 담당자 IP만 허용

서비스 등록

- ☐ systemd 서비스 등록 완료 (pg-audit-proxy 또는 pg-audit-sniffer)
- ☐ systemd 서비스 등록 완료 (audit-log-viewer)
- ☐ 서버 재부팅 후 자동 시작 확인 (`systemctl status`)

운영 중

- ☐ 로그 파일 주기적 백업 및 보관 기간 정책 수립 (6개월 이상 권장)
- ☐ `python3 log_viewer.py --verify` 또는 웹 `/verify` 로 무결성 정기 검증
- ☐ 로그 파티션 용량 모니터링 (200 MB/일 × 보관일수 추산)

☐ 웹 뷰어(8888) 방화벽 정기 점검

10. 보안 고려사항

웹 뷰어 접근 통제

웹 뷰어(`log_viewer_web.py`)는 **인증 기능이 없다**. 반드시 아래 조치를 취한다.

항목	조치
방화벽	8888 포트를 조회 담당자 IP 대역만 허용
내부망 전용	인터넷 노출 절대 금지
강화 권고	nginx + Basic Auth + HTTPS 앞단 구성 (선택)

로그 파일 보호

```
# 로그 폴더 권한 (감사 계정만 접근)
```

```
chmod 700 /opt/audit/logs/
```

```
chown audituser:audituser /opt/audit/logs/
```

append-only 설정 (삭제/수정 방지)

```
sudo chattr +a /opt/audit/logs/
```

SSL 암호화 미사용 시 주의

현재 Superset ↔ PostgreSQL 구간이 평문이므로 **같은 네트워크 세그먼트에서 패킷 도청 가능**하다. 내부망 전용 환경에서만 사용하고, 개인정보를 포함하는 데이터는 별도 암호화 정책을 수립한다.

무결성 해시 한계

`integrity_hash = sha256(log_id + start_time + sql + status)` 가 각 로그 항목에 포함된다. 로그 파일을 직접 편집하면 해시 불일치로 탐지되지만, 해시와 내용을 동시에 조작하면 탐지가 불가능하다. 중요 환경에서는 별도 WORM 저장소에 해시값을 보관한다.

11. 제약 사항

항목	내용
SSL/TLS	<code>sslmode=require</code> 환경에서 SQL 추출 불가 — 현재 환경(비암호화) 적합
바인드 변수 값	SQL 구조는 기록되나 실제 파라미터 값 추출 제한
대용량 SQL	여러 패킷으로 분할된 경우 일부 누락 가능 (방식 B만 해당)
result_data	text/varchar 계열만 추출. bytea, jsonb 등 바이너리 타입 제외
웹 뷰어 인증	기본 인증 없음 — 방화벽으로 접근 통제 필수
로그 암호화	로그 파일 평문 저장 — 파일 권한 + 별도 암호화 저장소 권고